A.扫雷

反悔贪心或者预处理后缀最小值

用一个变量存一下现在手中有多少个扫雷币，用一个变量存一下当前买了多少扫雷币（最后该变量的值就是答案）。

遍历数组，如果当前手中的钱能买扫雷币，就买。

用优先队列存一下所有的购买操作（多少钱买的，买了几个币），用大根堆来维护买的最贵的一次。

每次买完后，由花钱由多到少看一下之前所有的购买操作（不断看堆顶），如果之前买贵了，就“反悔”不买了，把之前买的钱（堆顶第一维乘第二维）拿回来，然后以当前更低的价格来买。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <queue>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int res = 0;
    int t = 0;
    int n;
    cin >> n;
    priority_queue<PII> pq;
    for(int i = 0; i < n; i++){
        int x;
        cin >> x;
        t++;
        if(t >= x){
            pq.push({x, t / x});
            res += t / x;
            t %= x;
        }
        while(pq.size() && x < pq.top().first){
            int val = pq.top().first, cnt = pq.top().second;
            pq.pop();
            res -= cnt;
            t += val * cnt;
            pq.push({x, t / x});
            res += t / x;
            t %= x;
        }
    }
    cout << res << endl;
    return 0;
}
```

预处理出a数组的后缀最小值数组suf，如果$a_i = suf_i$，说明此处就是一直到最后最便宜的地方，只要能买就一定在这买，能买多少买多少，遍历即可。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <queue>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;
const int N = 2e5 + 10;

int a[N], suf[N];

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t = 0;
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
    suf[n] = a[n];
    for(int i = n - 1; i >= 1; i--){
        suf[i] = min(suf[i + 1], a[i]);
    }
    int res = 0;
    for(int i = 1; i <= n; i++){
        t++;
        if(a[i] == suf[i]){
            res += t / a[i];
            t %= a[i];
        }
    }
    cout << res << endl;
    return 0;
}
```

B.优秀字符串

模拟。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <set>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    int res = 0;
```

```
    for(int i = 0; i < n; i++){
        string s;
        cin >> s;
        set<int> se;
        for(int j = 0; j < 4; j++) se.insert(s[j]);
        if(s.size() == 5 && s[2] == s[4] && se.size() == 4) res++;
    }
    cout << res << endl;
    return 0;
}
```

C.随机栈

快速幂求逆元、概率。

因为是n次插入n次取出，而最后要让序列是递增的，因此小的数肯定优先被取出。

由此可得：每次取出来的一定是当前还未取出的最小的数。 可以用一个小跟堆的优先队列来维护。

用一个变量$last$记录一下上次被取出的数，

对于放入操作：后放的数一定后取，所以如果$a_i < last$，概率一定为0；如果$a_i >= last$，就将其放入队列中。

对于取出操作：取出堆顶，如果堆顶比$last$小，概率一定为0，否则概率为$cnt(堆顶)/cnt(整个堆)$，分子直接乘，分母用快速幂求逆元转化成乘法，把每一步取出操作的概率累乘即可。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <queue>
#include <set>
#include <unordered_map>
#define endl '\n'
using namespace std;
using LL = long long;
typedef pair<int, int> PII;
const int N = 4e5 + 10;
const int mod = 998244353;

LL qmi(LL a, LL b, int p){
    LL res = 1 % p;
    while(b){
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

LL inv(LL n, int p){
    return qmi(n, p - 2, p);
}

int a[N];

```

```cpp
int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for(int i = 0; i < n * 2; i++){
        cin >> a[i];
    }
    priority_queue<int, vector<int>, greater<>> pq;
    int last = -1;
    LL res = 1;
    unordered_map<int, int> cnt;
    for(int i = 0; i < n * 2; i++){
        if(a[i] >= 0){
            if(a[i] >= last){
                pq.push(a[i]);
                cnt[a[i]]++;
            }
            else{
                cout << 0 << endl;
                return 0;
            }
        }
        else{
            if(pq.empty()){
                cout << 0 << endl;
                return 0;
            }
            res = res * cnt[pq.top()] % mod * inv(pq.size(), mod) % mod;
            last = pq.top();
            cnt[last]--;
            pq.pop();
        }
    }
    cout << res << endl;
    return 0;
}
```

D.排列与合数

暴力，从小到大遍历所有的排列。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <set>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;

bool is_not_prime(int n){
    if(n <= 1) return false;
    for(int i = 2; i <= n / i; i++){
        if(n % i == 0) return true;
    }
```

```
    return false;
};

void solve(){
    string s;
    cin >> s;
    sort(s.begin(), s.end());
    do{
        int n = stoi(s);
        if(s[0] == '0') continue;
        if(is_not_prime(n)){
            cout << n << endl;
            return;
        }
    }while(next_permutation(s.begin(), s.end()));
    cout << -1 << endl;
}

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

E.有效算法

二分答案。

题目中的式子：

$\left| a_i - x \right| \leq k \times b_i$

绝对值展开：

$- k \times b_i + x \le a_i \le k \times b_i + x $

要让所有的a_i都相等，也就是所有的a_i的范围有共同的一段交集，二分答案k，题目就转化成了：存在一个x，使得对于所有的a_i，都满足$- k \times b_i + x \le a_i \le k \times b_i + x $

由于x是任意整数，不好判断，而a_i都是已知的，可以将x控制在一个区间内，将绝对值换一种展开方式。

$-k \times b_i +a_i\le x \le k \times b_i + a_i$

这样的话，就可以遍历每一个a_i，取x的区间左端点（$-k \times b_i +a_i\le x$）的最大值和右端点（$k \times b_i + a_i$）的最小值，如果前者小于等于后者，就说明x是存在的，$check$函数返回$true$，否则x不存在，$check$函数返回$false$。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <cmath>
#include <set>
```

```cpp
#define endl '\n'
using namespace std;
using LL = long long;
typedef pair<int, int> PII;
const int N = 3e5 + 10;

int n;
int a[N], b[N];

bool check(int k){
    LL lmax = -1e18, rmin = 1e18;
    for(int i = 0; i < n; i++){
        lmax = max(lmax, -(LL)k * b[i] + a[i]);
        rmin = min(rmin, (LL)k * b[i] + a[i]);
    }
    if(lmax <= rmin) return true;
    return false;
}

void solve(){
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> a[i];
    }
    for(int i = 0; i < n; i++){
        cin >> b[i];
    }
    int l = 0, r = 1e9;
    while(l < r){
        int mid = l + r >> 1;
        if(check(mid)) r = mid;
        else l = mid + 1;
    }
    cout << l << endl;
}

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

F.2025

按题意模拟，需要注意：$sqrt$返回值是浮点数。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <cmath>
#include <set>
#define endl '\n'
```

```cpp
using namespace std;
typedef pair<int, int> PII;

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    int sum = 0;
    int t = n;
    while(t){
        sum += t % 10;
        t /= 10;
    }
    if((int)sqrt(n) * (int)sqrt(n) == n && (int)sqrt(sum) * (int)sqrt(sum) == sum){
        cout << "Yes" << endl;
    }
    else cout << "No" << endl;
    return 0;
}
```

G.幻形之路

三个bfs。

第一个：从起点开始bfs，标记起点不走障碍能走到的所有点。

第二个：从终点开始bfs，标记终点不走障碍能走到的所有点。

第三个：多源bfs，将起点和其标记的点都入队，从这些点开始走，忽略障碍，第一次走到被终点标记的点时的步数即要求的最小k值。

需要注意题目中的范围，$\sum nm \le 10^6$，但开数组肯定要开1000×1000的数组，如果每次都$memset$整个数组的话，复杂度就是$1000 \times 1000 \times T$，必然会超时，所以可以用$for$循环/$vector$/$fill$...，每次只初始化用的部分。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <queue>
#include <cstring>
#define endl '\n'
using namespace std;
using LL = long long;
typedef pair<int, int> PII;
const int N = 1010;
const int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};
int n, m;
string g[N];
bool st1[N][N], st2[N][N];
int dist[N][N];

void bfs1(int sx, int sy, bool (&st)[N][N]){
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
```

```
            st[i][j] = false;
        }
    }
    queue<PII> q;
    q.push({sx, sy});
    st[sx][sy] = true;
    while(!q.empty()){
        auto [x, y] = q.front();
        q.pop();
        for(int i = 0; i < 4; i++){
            int a = x + dx[i], b = y + dy[i];
            if(a < 1 || b < 1 || a > n || b > m) continue;
            if(g[a][b] == '#') continue;
            if(st[a][b]) continue;
            st[a][b] = true;
            q.push({a, b});
        }
    }
}

int bfs2(){
    if(st1[n][m]) return 0;
    queue<PII> q;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            dist[i][j] = -1;
        }
    }
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            if(st1[i][j]){
                q.push({i, j});
                dist[i][j] = 0;
            }
        }
    }
    while(!q.empty()){
        auto [x, y] = q.front();
        q.pop();
        for(int i = 0; i < 4; i++){
            int a = x + dx[i], b = y + dy[i];
            if(a < 1 || b < 1 || a > n || b > m) continue;
            if(~dist[a][b]) continue;
            dist[a][b] = dist[x][y] + 1;
            if(st2[a][b]) return dist[a][b] - 1;
            q.push({a, b});
        }
    }
}

void solve(){
    cin >> n >> m;
    for(int i = 1; i <= n; i++){
        cin >> g[i];
        g[i] = " " + g[i];
    }
```

```
    bfs1(1, 1, st1);
    bfs1(n, m, st2);
    cout << bfs2() << endl;
}

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

H.直径与最大独立集

构造。

通过手写模拟或者打表可以看出来有规律。

当$n \le 4$的情况是平凡的。

当$n > 4$时，一定有解。构造的方法有很多，下面给出一种可行的构造，大体思路是：构造一条链加上一个菊花图的形状。

令$t = \lfloor \frac{n + 2}{3} \rfloor + 2$

当$n \% 3 \not= 1$时：

$\forall i \in [2, t]$，与1连边；$\forall i \in [t, n)$，与$i + 1$连边。

当$n \% 3 = 1$时：

$\forall i \in [2, t]$，与1连边；$\forall i \in [t, n - 1)$，与$i + 1$连边，3与n连边。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <queue>
#include <cstring>
#define endl '\n'
using namespace std;
using LL = long long;
typedef pair<int, int> PII;

void solve(){
    int n;
    cin >> n;
    if(n == 2) cout << 1 << ' ' << 2 << endl;
    else if(n == 3){
        cout << 1 << ' ' << 2 << endl;
        cout << 2 << ' ' << 3 << endl;
    }
    else if(n == 4) cout << -1 << endl;
    else{
        int t = (n + 2) / 3 + 2;
        if (n % 3 != 1){
            for(int i = 2; i <= t; i++){
```

```
                cout << 1 << ' ' << i << endl;
            }
            for(int i = t; i < n; i++){
                cout << i << ' ' << i + 1 << endl;
            }
        }
        else{
            for(int i = 2; i <= t; i++){
                cout << 1 << ' ' << i << endl;
            }
            for(int i = t; i < n - 1; i++){
                cout << i << ' ' << i + 1 << endl;
            }
            cout << 3 << ' ' << n << endl;
        }
    }
}

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

I.Ring Trick

模拟。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <cmath>
#include <set>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int holes_cnt[26] = {1, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0};
    string s;
    cin >> s;
    int n = s.size();
    int res = 0;
    for(int k = 0; k < 26; k++){
        int sum = 0;
        for(int i = 0; i < n; i++){
            sum += holes_cnt[(s[i] - 'A' + k) % 26];
        }
        res = max(res, sum);
    }
```

```
    cout << res << endl;
    return 0;
}
```

J.川陀航空学院

并查集判环、并查集求联通块数量。

答案 = 多余的边的数量 + 少的边的数量 。

多余的边：如果加边之前，A和B就是连通的，那A-B这条边就是多余的。

少的边：所有边加完之后，如果有cnt个联通块，那就还需要加$cnt - 1$条边给他们连起来。

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
#include <cmath>
#include <set>
#define endl '\n'
using namespace std;
typedef pair<int, int> PII;
const int N = 1e6 + 10;

int fa[N];

int find(int x){
    if(fa[x] != x) fa[x] = find(fa[x]);
    return fa[x];
}

int main(){
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int n, m;
    cin >> n >> m;
    int cnt1 = 0, cnt2 = 0;
    for(int i = 1; i <= n; i++) fa[i] = i;
    while(m--){
        int a, b;
        cin >> a >> b;
        a = find(a), b = find(b);
        if(a != b) fa[a] = b;
        else cnt1++;
    }
    for(int i = 1; i <= n; i++){
        if(fa[i] == i) cnt2++;
    }
    cout << cnt1 + cnt2 - 1 << endl;
    return 0;
}
```